

18 AQE (Adaptive Query Execution)

Adaptive Query Execution (AQE) is a dynamic query optimization framework in Apache Spark 3.0+ that re-optimizes and adjusts query plans based on runtime data statistics. It improves performance by automatically coalescing small partitions, switching join strategies, and handling data skew, reducing the need for manual tuning.

1- Partition skew

What is partition skew

Partition skew happens when **data is unevenly distributed across partitions**, so:

- A few partitions are **very large**
- Most partitions are **small or empty**

Result:

- Some tasks run very slow
- Others finish quickly
- Overall job waits for the slowest task
- can lead to out of memory error

```
df_skewed = df_orders.withColumn("product_id", when(col("product_id") <= 90, 1).otherwise(col("product_id")))
# df_skewed.groupBy("product_id").agg(count("*")).show()

df_skewed.join(df_products, "product_id", "inner").write.mode("overwrite").format("noop").save()
```

Solution : Salting

Salting is a technique to **fix data skew** by **artificially spreading a hot key across multiple partitions**.

```
so for product id 1 we will modify it to have  
5 million with value key
```

```
1 million key_1  
1 million key_2  
1 million key_3  
1 million key_4  
1 million key_5
```

```
now they will distributed accross 5 parttions intead of one  
then we need to write a logic to combine the result for eac  
h key
```

2- No of shuffle partitions :

by default 200 and some partitions can have empty or very less data.

this can be modified

```
#spark.conf.get("spark.sql.shuffle.partitions")
```

3- Static execution plan

Once a plan is created, Spark **cannot change it** even if it is bad.

If estimates are wrong, Spark may pick:

- Sort Merge Join instead of Broadcast Join
- Very slow execution

Adaptive Query Execution

1- Dynamically coalescing number of shuffle partitions

it will remove empty partitions and it can also coalesce partitions which have less data

```
from pyspark.sql.types import *
from pyspark.sql.functions import *
orders_schema_struct = StructType(
[
StructField("order_id", IntegerType(), False)
, StructField("customer_id", IntegerType(), False)
, StructField("product_id", IntegerType(), False)
, StructField("unit_price", FloatType(), False)
, StructField("order_date", DateType(), False)
, StructField("order_status", StringType(), False)
, StructField("state", StringType(), False)
, StructField("quantity", IntegerType(), False)
]
)

df_orders = spark.read.csv("/data/orders_lgb.csv" , header
= True , schema = orders_schema_struct)

# first local aggregation in group by and then shuffle
df_orders.groupBy("order_status").count().write.format("noop").mode("overwrite").save()
orders_df.groupBy("state", "order_status", "order_date").count().write.format("noop").mode("overwrite").save()

orders_df.groupBy("customer_id").count().write.format("noop").mode("overwrite").save()
# adding partition id column to verify local aggregation
from pyspark.sql.functions import spark_partition_id
df_with_pid = orders_df.withColumn("partition_id", spark_partition_id())
df_with_pid.groupBy("partition_id") \
    .agg(countDistinct("customer_id").alias("distinct_
```

```
customers")) \
    .show()
```

2- Dynamically handling partition skew

```
from pyspark.sql.types import *
from pyspark.sql.functions import *
orders_schema_struct = StructType(
[
    StructField("order_id", IntegerType(), False)
    , StructField("customer_id", IntegerType(), False)
    , StructField("product_id", IntegerType(), False)
    , StructField("unit_price", FloatType(), False)
    , StructField("order_date", DateType(), False)
    , StructField("order_status", StringType(), False)
    , StructField("state", StringType(), False)
    , StructField("quantity", IntegerType(), False)
]
)

df_orders = spark.read.csv("/data/orders_1gb.csv" , header
= True , schema = orders_schema_struct)

product_schema = 'product_id int , cost_price float'

df_products = spark.read.csv("/data/products_12mb.csv" , header = True , schema = product_schema)
df_orders.show()
df_products.show()

df_skewed = df_orders.withColumn("product_id", when(col("product_id")<=90 , 1 ).otherwise(col("product_id")))
df_skewed.groupBy("product_id").agg(count("*")).show()
df_skewed.join(df_products, "product_id", "inner").write.mode("overwrite").format("noop").save()
```

3- Dynamically switching join strategies

during shuffle it calculates runtime stats and get the latest view of data count/size , skewness etc

Plan before and after AQE enabled

```
from pyspark.sql.types import *
from pyspark.sql.functions import *
orders_schema_struct = StructType(
[
StructField("order_id",IntegerType(),False)
,StructField("customer_id",IntegerType(),False)
,StructField("product_id",IntegerType(),False)
,StructField("unit_price",FloatType(),False)
,StructField("order_date",DateType(),False)
,StructField("order_status",StringType(),False)
,StructField("state",StringType(),False)
,StructField("quantity",IntegerType(),False)
]
)

df_orders = spark.read.csv("/data/orders_1gb.csv" , header
= True ,  schema = orders_schema_struct)

product_schema = 'product_id int , cost_price float'

df_products = spark.read.csv("/data/products_12mb.csv" , header = True ,  schema = product_schema)
df_orders.show()
df_products.show()

df_skewed = df_orders.withColumn("product_id",  when(col("product_id")<=90 , 1 ).otherwise(col("product_id")))
# df_skewed.groupBy("product_id").agg(count("*")).show()
df_products_filtered = df_products.filter(col("product_id")<=100)
#df_skewed.join(df_products, "product_id", "inner").write.mode("overwrite").format("noop").save()
```

```
df_skewed.join(df_products_filtered, "product_id", "inner").write.mode("overwrite").format("noop").save()
```

configs

```
spark.conf.set("spark.sql.adaptive.enabled", True)
```